

FINAL PROJECT REPORT

Bachelor of Technology in Computer Science and Engineering

Class BTech CSE 25–29

PROJECT TITLE

SkyLine Airlines(Flight Reservation and Seat Allotment System)

Submitted By	Sanjana Bashya
Roll No:	150096725113
Cohort:	Larry Page
Subject	DBMS-SQL & NOSQL

1. Project Overview

The SkyLine Airlines Flight Reservation and Seat Allotment System is a relational database management system designed to manage airline flights, airports, passengers, seats and bookings.

The main purpose of the project is to maintain accurate flight reservation information and prevent two passengers from being assigned the same seat on the same flight.

The system stores flight schedules, airport information, passenger details, available seats and booking records. SQL constraints, primary keys, foreign keys and a unique constraint are used to maintain data integrity.

The project demonstrates important DBMS concepts such as:

- Relational database design, entities and attributes
- Primary keys, foreign keys, NOT NULL, UNIQUE and CHECK constraints
- Normalization (1NF, 2NF, 3NF)
- SQL DDL, DML and SELECT queries with WHERE conditions
- JOIN operations, LEFT JOIN and CROSS JOIN
- Subqueries and NOT IN
- GROUP BY, COUNT() and aggregate functions
- Stored functions and constraint validation
- Seat availability checking and flight occupancy calculation

The system provides a structured way to manage airline reservation information and demonstrates how SQL can be used to solve a real-world booking problem.

2. Problem Statement

SkyLine Airlines requires a database system for managing flight reservations and seat allocation. The system must:

- Store airport information, flight details and passenger information.
- Maintain a list of available seats and store passenger bookings.
- Prevent the same seat from being assigned twice on the same flight.
- Allow staff to check available seats and display passenger booking information.
- Calculate occupied seats for each flight.
- Validate invalid passenger, flight and seat information.
- Maintain referential integrity using foreign keys.
- Generate useful reports using SQL queries.

3. Project Objectives

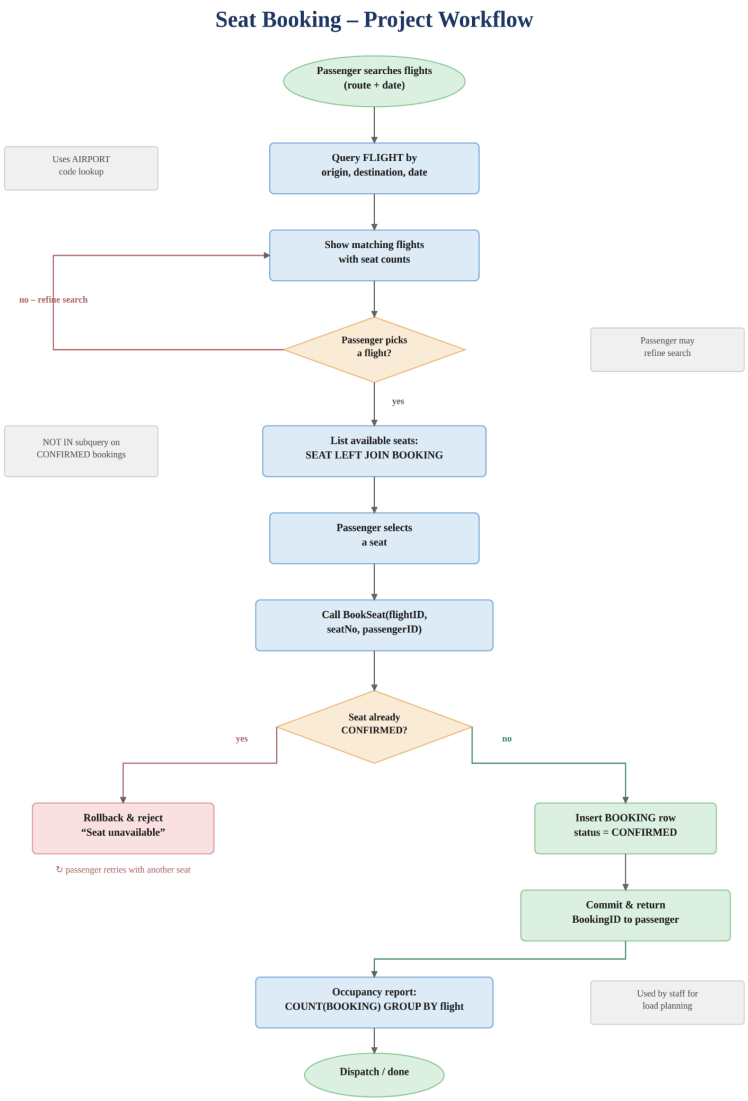
- To design a relational database for an airline reservation system and identify entities and relationships.
- To create tables using SQL with appropriate primary keys and foreign keys.
- To maintain data integrity using constraints and normalize the database.

- To insert realistic sample records and retrieve information using SQL queries.
- To combine data using JOIN operations and find available seats using subqueries.
- To calculate flight occupancy using aggregate functions.
- To implement a BookSeat stored function and validate successful and unsuccessful bookings.
- To prevent duplicate seat allocation and demonstrate practical DBMS concepts through a real-world application.

4. Technologies and Tools Used

Component	Technology
Database	PostgreSQL
Query Language	SQL
Interface	PostgreSQL Terminal / psql
Database Name	skyline_airlines
Database Type	Relational Database
Main Tables	Airport, Flight, Passenger, Seat, Booking
Advanced SQL	JOIN, Subquery, GROUP BY, Aggregate Functions
Database Logic	Stored Function
Operating Environment	macOS

5. Project Workflow Diagram



[Insert Project Workflow Diagram here]

6. Database Design

The database contains five main entities: Airport, Flight, Passenger, Seat and Booking.

6.1 Airport – Primary Key: AirportID. Attributes: AirportID, AirportName, City, Country. Stores information about airports used as source and destination locations.

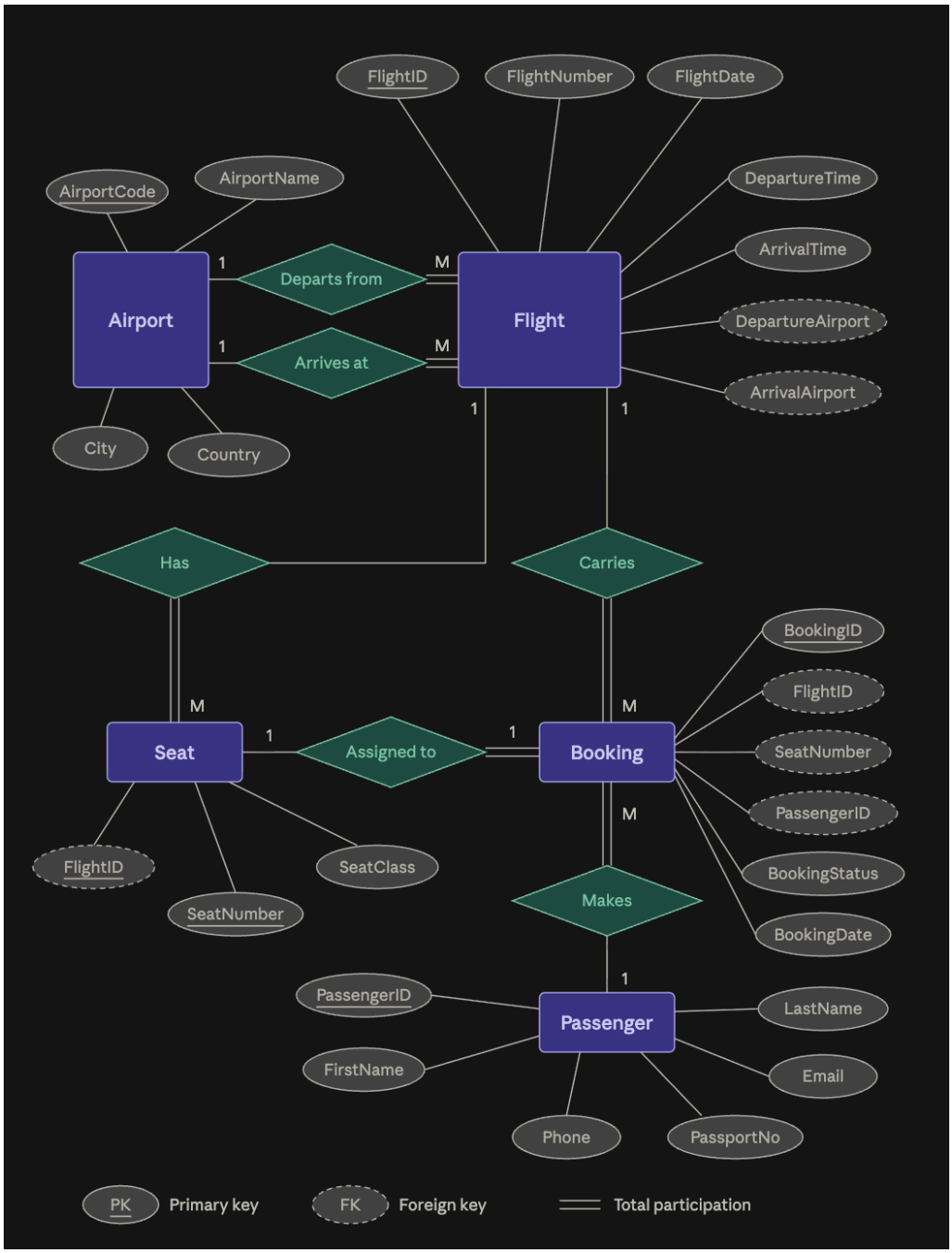
6.2 Flight – Primary Key: FlightID. Foreign Keys: SourceAirport, DestinationAirport. Attributes: FlightID, FlightNumber, SourceAirport, DestinationAirport, FlightDate, DepartureTime. Stores information about scheduled flights.

6.3 Passenger – Primary Key: PassengerID. Attributes: PassengerID, PassengerName, Email, Phone. Stores passenger details.

6.4 Seat – Primary Key: SeatNumber. Stores the seats available in the aircraft.

6.5 Booking – Primary Key: BookingID. Foreign Keys: FlightID, SeatNumber, PassengerID. Attributes: BookingID, FlightID, SeatNumber, PassengerID, BookingStatus. Connects passengers, flights and seats.

7. ER Diagram



8. Relational Schema

Airport (AirportID PK, AirportName, City, Country)

Flight (FlightID PK, FlightNumber,
SourceAirport FK, DestinationAirport FK,
FlightDate, DepartureTime)

Passenger (PassengerID PK, PassengerName, Email, Phone)

Seat (SeatNumber PK)

Booking (BookingID PK, FlightID FK, SeatNumber FK,
PassengerID FK, BookingStatus)
UNIQUE (FlightID, SeatNumber)

9. DDL – Table Creation

The database tables were created using SQL Data Definition Language (DDL). The project uses PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE and CHECK constraints. Primary keys uniquely identify records and foreign keys establish relationships between tables. The UNIQUE constraint on (FlightID, SeatNumber) prevents the same seat from being assigned twice on the same flight.

9.1 Table Verification

```
skyline_airlines=# \dt
              List of tables
 Schema |   Name    | Type  |     Owner
--------+-----------+-------+---------------
 public | airport   | table | sanjanabashya
 public | booking   | table | sanjanabashya
 public | flight    | table | sanjanabashya
 public | passenger | table | sanjanabashya
 public | seat      | table | sanjanabashya
(5 rows)
```

9.2 Flight Table Structure

```
skyline_airlines=# \d Flight
                          Table "public.flight"
       Column       |          Type          | Collation | Nullable | Default
--------------------+------------------------+-----------+----------+---------
 flightid           | integer                |           | not null |
 flightnumber       | character varying(20)  |           | not null |
 sourceairport      | integer                |           | not null |
 destinationairport | integer                |           | not null |
 flightdate         | date                   |           | not null |
 departuretime      | time without time zone |           | not null |
Indexes:
    "flight_pkey" PRIMARY KEY, btree (flightid)
Foreign-key constraints:
    "flight_destinationairport_fkey" FOREIGN KEY (destinationairport) REFERENCES airport(airportid)
    "flight_sourceairport_fkey" FOREIGN KEY (sourceairport) REFERENCES airport(airportid)
Referenced by:
    TABLE "booking" CONSTRAINT "booking_flightid_fkey" FOREIGN KEY (flightid) REFERENCES flight(flightid)
```

9.3 Passenger Table Structure

```
skyline_airlines=# \d Passenger
                      Table "public.passenger"
    Column     |          Type          | Collation | Nullable | Default
---------------+------------------------+-----------+----------+---------
 passengerid   | integer                |           | not null |
 passengername | character varying(100) |           | not null |
 email         | character varying(100) |           | not null |
 phone         | character varying(15)  |           | not null |
Indexes:
    "passenger_pkey" PRIMARY KEY, btree (passengerid)
Referenced by:
    TABLE "booking" CONSTRAINT "booking_passengerid_fkey" FOREIGN KEY (passengerid) REFERENCES passenger(passengerid)
```

9.4 Seat Table Structure

```
skyline_airlines=# \d Seat
                       Table "public.seat"
   Column   |         Type          | Collation | Nullable | Default
------------+-----------------------+-----------+----------+---------
 seatnumber | character varying(5) |           | not null |
Indexes:
    "seat_pkey" PRIMARY KEY, btree (seatnumber)
Referenced by:
    TABLE "booking" CONSTRAINT "booking_seatnumber_fkey" FOREIGN KEY (seatnumber) REFERENCES seat(seatnumber)
```

9.5 Booking Table Structure

```
skyline_airlines=# \d Booking
                        Table "public.booking"
    Column     |         Type          | Collation | Nullable | Default
---------------+-----------------------+-----------+----------+---------
 bookingid     | integer               |           | not null |
 flightid      | integer               |           | not null |
 seatnumber    | character varying(5)  |           | not null |
 passengerid   | integer               |           | not null |
 bookingstatus | character varying(20) |           | not null |
Indexes:
    "booking_pkey" PRIMARY KEY, btree (bookingid)
    "booking_flightid_seatnumber_key" UNIQUE CONSTRAINT, btree (flightid, seatnumber)
Foreign-key constraints:
    "booking_flightid_fkey" FOREIGN KEY (flightid) REFERENCES flight(flightid)
    "booking_passengerid_fkey" FOREIGN KEY (passengerid) REFERENCES passenger(passengerid)
    "booking_seatnumber_fkey" FOREIGN KEY (seatnumber) REFERENCES seat(seatnumber)
```

9.6 Airport Table Structure

```
skyline_airlines=# \d Airport
                    Table "public.airport"
   Column    |          Type          | Collation | Nullable | Default
-------------+------------------------+-----------+----------+---------
 airportid   | integer                |           | not null |
 airportname | character varying(100) |           | not null |
 city        | character varying(50)  |           | not null |
 country     | character varying(50)  |           | not null |
Indexes:
    "airport_pkey" PRIMARY KEY, btree (airportid)
Referenced by:
    TABLE "flight" CONSTRAINT "flight_destinationairport_fkey" FOREIGN KEY (destinationairport) REFERENCES airport(airportid)
    TABLE "flight" CONSTRAINT "flight_sourceairport_fkey" FOREIGN KEY (sourceairport) REFERENCES airport(airportid)
```

10. DML – Sample Data

Sample records were inserted into the Airport, Flight, Passenger, Seat and Booking tables. The outputs below show the initial sample data (four bookings). A fifth booking (305) is added later using the BookSeat function.

10.1 Airport Table Output

```
skyline_airlines=# SELECT * FROM Airport;
 airportid |                    airportname                    |   city    | country
-----------+---------------------------------------------------+-----------+---------
         1 | Chhatrapati Shivaji Maharaj International Airport | Mumbai    | India
         2 | Indira Gandhi International Airport               | Delhi     | India
         3 | Kempegowda International Airport                  | Bangalore | India
         4 | Rajiv Gandhi International Airport                | Hyderabad | India
         5 | Chennai International Airport                     | Chennai   | India
(5 rows)
```

10.2 Flight Table Output

```
skyline_airlines=# SELECT * FROM Flight;
 flightid | flightnumber | sourceairport | destinationairport | flightdate | departuretime
----------+--------------+---------------+--------------------+------------+---------------
      201 | SK101        |             1 |                  2 | 2026-10-05 | 09:00:00
      202 | SK102        |             2 |                  3 | 2026-10-05 | 11:30:00
      203 | SK103        |             1 |                  4 | 2026-10-06 | 14:00:00
      204 | SK104        |             3 |                  5 | 2026-10-06 | 16:30:00
      205 | SK105        |             4 |                  1 | 2026-10-07 | 19:00:00
(5 rows)
```

10.3 Passenger Table Output

```
skyline_airlines=# SELECT * FROM Passenger;
 passengerid | passengername |       email        |   phone
-------------+---------------+--------------------+------------
         101 | Aanya         | Aanya@gmail.com    | 9876543210
         102 | Aman          | aman@gmail.com     | 9876543211
         103 | Mukesh        | mukesh@gmail.com   | 9876543212
         105 | Ananya        | ananya@gmail.com   | 9876543214
(4 rows)
```

10.4 Seat Table Output

```
skyline_airlines=# SELECT * FROM Seat;
 seatnumber
------------
 1A
 1B
 1C
 1D
 2A
 2B
 2C
 2D
 3A
 3B
(10 rows)
```

10.5 Booking Table Output (initial data)

```
skyline_airlines=# SELECT * FROM Booking;
 bookingid | flightid | seatnumber | passengerid | bookingstatus
-----------+----------+------------+-------------+---------------
       301 |      201 | 1A         |         101 | Confirmed
       302 |      201 | 1B         |         102 | Confirmed
       303 |      202 | 1A         |         103 | Confirmed
       304 |      203 | 2A         |         105 | Confirmed
(4 rows)
```

11. Normalization

Normalization is used to reduce data redundancy and prevent insertion, update and deletion anomalies.

First Normal Form – 1NF

- Each table has atomic attributes and there are no repeating groups.
- Each field contains a single value; passenger details are stored separately instead of storing multiple passengers in one field.

Second Normal Form – 2NF The database follows 2NF because non-key attributes depend on the complete primary key. Passenger, flight, airport and booking information are each stored in their own tables, which removes the partial dependency of repeating passenger contact details on every booking row.

Third Normal Form – 3NF Non-key attributes depend only on the primary key of their table and transitive dependencies are avoided. For example, airport name and city are stored in the Airport table instead of being repeated for every flight.

12. SQL Queries and Outputs

12.1 Flight Route Query

```
skyline_airlines=# SELECT
    f.FlightNumber,
    a1.AirportName AS SourceAirport,
    a2.AirportName AS DestinationAirport,
    f.FlightDate,
    f.DepartureTime
FROM Flight f
JOIN Airport a1
    ON f.SourceAirport = a1.AirportID
JOIN Airport a2
    ON f.DestinationAirport = a2.AirportID
ORDER BY f.FlightDate;
 flightnumber |                  sourceairport                  |               destinationairport                | flightdate | departuretime
--------------+-------------------------------------------------+-------------------------------------------------+------------+---------------
 SK101        | Chhatrapati Shivaji Maharaj International Airport | Indira Gandhi International Airport             | 2026-10-05 | 09:00:00
 SK102        | Indira Gandhi International Airport             | Kempegowda International Airport                | 2026-10-05 | 11:30:00
 SK103        | Chhatrapati Shivaji Maharaj International Airport | Rajiv Gandhi International Airport              | 2026-10-06 | 14:00:00
 SK104        | Kempegowda International Airport                | Chennai International Airport                   | 2026-10-06 | 16:30:00
 SK105        | Rajiv Gandhi International Airport              | Chhatrapati Shivaji Maharaj International Airport | 2026-10-07 | 19:00:00
(5 rows)
```

12.2 Search Flights by Date

```
skyline_airlines=# SELECT
    FlightNumber,
    FlightDate,
    DepartureTime
FROM Flight
WHERE FlightDate = '2026-10-05'
ORDER BY DepartureTime;
 flightnumber | flightdate | departuretime
--------------+------------+---------------
 SK101        | 2026-10-05 | 09:00:00
 SK102        | 2026-10-05 | 11:30:00
(2 rows)
```

12.3 Complete Booking Information using JOIN

```
skyline_airlines=# SELECT
    b.BookingID,
    f.FlightNumber,
    f.FlightDate,
    p.PassengerName,
    b.SeatNumber,
    b.BookingStatus
FROM Booking b
JOIN Flight f
    ON b.FlightID = f.FlightID
JOIN Passenger p
    ON b.PassengerID = p.PassengerID
ORDER BY f.FlightDate, b.BookingID;
 bookingid | flightnumber | flightdate | passengername | seatnumber | bookingstatus
-----------+--------------+------------+---------------+------------+---------------
       301 | SK101        | 2026-10-05 | Aanya         | 1A         | Confirmed
       302 | SK101        | 2026-10-05 | Aman          | 1B         | Confirmed
       303 | SK102        | 2026-10-05 | Mukesh        | 1A         | Confirmed
       305 | SK101        | 2026-10-05 | Mukesh        | 2A         | Confirmed
       304 | SK103        | 2026-10-06 | Ananya        | 2A         | Confirmed
(5 rows)
```

12.4 Passengers Without Bookings (LEFT JOIN)

```
skyline_airlines=# SELECT
    p.PassengerID,
    p.PassengerName
FROM Passenger p
LEFT JOIN Booking b
    ON p.PassengerID = b.PassengerID
WHERE b.BookingID IS NULL;
 passengerid | passengername
-------------+---------------
(0 rows)
```

12.5 Available Seat Search (NOT IN subquery)

```
skyline_airlines=# SELECT
    s.SeatNumber
FROM Seat s
WHERE s.SeatNumber NOT IN (
    SELECT b.SeatNumber
    FROM Booking b
    WHERE b.FlightID = 201
)
ORDER BY s.SeatNumber;
 seatnumber
------------
 1C
 1D
 2B
 2C
 2D
 3A
 3B
(7 rows)
```

12.6 Available Seats for a Flight and Date (JOIN + NOT IN)

```
skyline_airlines=# SELECT s.SeatNumber
FROM Seat s
LEFT JOIN Booking b
    ON s.SeatNumber = b.SeatNumber AND b.FlightID = 201
WHERE s.SeatNumber NOT IN (
    SELECT b2.SeatNumber
    FROM Booking b2
    JOIN Flight f ON b2.FlightID = f.FlightID
    WHERE f.FlightID = 201 AND f.FlightDate = '2026-10-05'
);
 seatnumber
------------
 1C
 1D
 2B
 2C
 2D
 3A
 3B
(7 rows)
```

Combines a LEFT JOIN between Seat and Booking with a NOT IN subquery that filters bookings by flight and (7 rows)

12.7 Aggregate Function – Occupancy Report

```
skyline_airlines=# SELECT
    f.FlightNumber,
    COUNT(b.BookingID) AS OccupiedSeats
FROM Flight f
LEFT JOIN Booking b
    ON f.FlightID = b.FlightID
GROUP BY f.FlightNumber
ORDER BY f.FlightNumber;
 flightnumber | occupiedseats
--------------+---------------
 SK101        |             3
 SK102        |             1
 SK103        |             1
 SK104        |             0
 SK105        |             0
(5 rows)
```

12.8 Occupancy Grouped by FlightID

```
skyline_airlines=# SELECT FlightID, COUNT(*) AS OccupiedSeats
FROM Booking
GROUP BY FlightID
ORDER BY FlightID;
 flightid | occupiedseats
----------+---------------
      201 |             3
      202 |             1
      203 |             1
(3 rows)
```

12.9 CROSS JOIN – Available Seats

```
skyline_airlines=# SELECT f.FlightNumber, f.FlightDate, s.SeatNumber
FROM Flight f
CROSS JOIN Seat s
WHERE f.FlightID = 201
  AND s.SeatNumber NOT IN (
      SELECT b.SeatNumber FROM Booking b WHERE b.FlightID = f.FlightID
  );
 flightnumber | flightdate | seatnumber
--------------+------------+------------
 SK101        | 2026-10-05 | 1C
 SK101        | 2026-10-05 | 1D
 SK101        | 2026-10-05 | 2B
 SK101        | 2026-10-05 | 2C
 SK101        | 2026-10-05 | 2D
 SK101        | 2026-10-05 | 3A
 SK101        | 2026-10-05 | 3B
(7 rows)
```

13. BookSeat Stored Function

13.1 Function Verification

```
skyline_airlines=# \df BookSeat
                                  List of functions
 Schema |   Name   | Result data type |               Argument data types                | Type
--------+----------+------------------+--------------------------------------------------+------
 public | bookseat | void             | fid integer, sno character varying, pid integer | func
(1 row)
```

13.2 Successful Booking Test

```
CREATE FUNCTION
skyline_airlines=# SELECT BookSeat(201,'2A',103);
 bookseat
----------

(1 row)

skyline_airlines=# SELECT * FROM Booking WHERE FlightID=201;
 bookingid | flightid | seatnumber | passengerid | bookingstatus
-----------+----------+------------+-------------+---------------
       301 |      201 | 1A         |         101 | Confirmed
       302 |      201 | 1B         |         102 | Confirmed
       305 |      201 | 2A         |         103 | Confirmed
(3 rows)
```

13.3 Unique Seat Constraint Test

```
skyline_airlines=# SELECT
    b.BookingID,
    f.FlightNumber,
    f.FlightDate,
    p.PassengerName,
    b.SeatNumber,
    b.BookingStatus
FROM Booking b
JOIN Flight f
    ON b.FlightID = f.FlightID
JOIN Passenger p
    ON b.PassengerID = p.PassengerID
ORDER BY b.BookingID DESC
LIMIT 1;
 bookingid | flightnumber | flightdate | passengername | seatnumber | bookingstatus
-----------+--------------+------------+---------------+------------+---------------
       305 | SK101        | 2026-10-05 | Mukesh        | 2A         | Confirmed
(1 row)
```

13.4 BookSeat Rejecting a Taken Seat

```
skyline_airlines=# INSERT INTO Booking
    (BookingID, FlightID, SeatNumber, PassengerID, BookingStatus)
VALUES
    (998, 201, '1A', 105, 'Confirmed');
ERROR:  duplicate key value violates unique constraint "booking_flightid_seatnumber_key"
DETAIL:  Key (flightid, seatnumber)=(201, 1A) already exists.
skyline_airlines=#
```

14. Final Database Verification

```
skyline_airlines=# SELECT
    (SELECT COUNT(*) FROM Airport) AS Airports,
    (SELECT COUNT(*) FROM Flight) AS Flights,
    (SELECT COUNT(*) FROM Passenger) AS Passengers,
    (SELECT COUNT(*) FROM Seat) AS Seats,
    (SELECT COUNT(*) FROM Booking) AS Bookings;
 airports | flights | passengers | seats | bookings
----------+---------+------------+-------+----------
        5 |       5 |          4 |    10 |        5
(1 row)
```

15. Data Integrity

Data integrity means maintaining accurate and consistent information in the database. The project maintains it through:

- Primary keys, foreign keys, NOT NULL, UNIQUE and CHECK constraints
- Stored function validation

The UNIQUE (FlightID, SeatNumber) constraint is particularly important because it prevents duplicate seat allocation.

16. Referential Integrity

```
skyline_airlines=# INSERT INTO Booking VALUES (990, 999, '1C', 101, 'Confirmed');   -- invalid flight
INSERT INTO Booking VALUES (991, 201, '9Z', 101, 'Confirmed');   -- invalid seat
INSERT INTO Booking VALUES (992, 201, '1C', 999, 'Confirmed');   -- invalid passenger
ERROR:  insert or update on table "booking" violates foreign key constraint "booking_flightid_fkey"
DETAIL:  Key (flightid)=(999) is not present in table "flight".
ERROR:  insert or update on table "booking" violates foreign key constraint "booking_seatnumber_fkey"
DETAIL:  Key (seatnumber)=(9Z) is not present in table "seat".
ERROR:  insert or update on table "booking" violates foreign key constraint "booking_passengerid_fkey"
DETAIL:  Key (passengerid)=(999) is not present in table "passenger".
```

17. Seat Allocation Logic

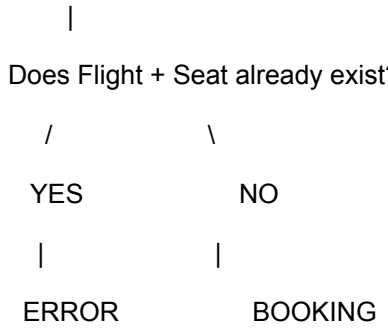
Passenger selects Flight

|

Passenger selects Seat

|

Check Flight -> Check Passenger -> Check Seat



This ensures that two passengers cannot receive the same seat on the same flight.

18. Advantages of the System

- Prevents duplicate seat allocation.
- Maintains passenger, flight, airport, seat and booking records.
- Provides available-seat and flight occupancy information.
- Maintains referential integrity and reduces redundancy through normalization.
- Provides reusable booking logic through a stored function.
- Supports reporting using SQL queries.

19. Limitations

The current academic implementation focuses on database functionality. It does not include an online payment gateway, user authentication, a staff web interface, email/SMS integration, real-time aircraft tracking, ticket generation, production-level concurrency management or cloud deployment.

20. Future Scope

- Web-based booking interface and mobile application
- User login, authentication and online payment integration
- E-ticket generation, email and SMS notifications
- Real-time seat availability and flight tracking
- Flight cancellation, baggage management, passenger check-in
- Loyalty/reward system, admin and analytics dashboards
- Cloud database deployment
- NoSQL integration for selected high-volume application data

21. Conclusion

The SkyLine Airlines Flight Reservation and Seat Allotment System demonstrates how a relational database can be designed and implemented for a real-world airline reservation scenario.

The project uses PostgreSQL and SQL to manage airports, flights, passengers, seats and bookings. Primary keys, foreign keys, NOT NULL constraints and the unique (FlightID, SeatNumber) constraint help maintain data consistency and prevent duplicate seat allocation.

SQL JOINs, subqueries, aggregate functions and GROUP BY operations retrieve meaningful information such as passenger booking details, available seats and flight occupancy. The BookSeat stored function shows how database-side logic can validate booking requests before creating a reservation.